

Universidad Autónoma de la Ciudad de México

Licenciatura en Ingeniería de Software

Arquitectura de Computadoras

Práctica 05

Producto Punto de Vectores en Ensamblador x86

Alumno: Edson Joel Carrera Avila

Grupo: 302

Profesor: Prof. Mtro. Omar Nieto Crisóstomo

Fecha: April 26, 2026

Contents

1	Introducción	2
2	Organización del proyecto	3
3	Código fuente	4
3.1	Código C++ (main.cpp)	4
3.2	Código ensamblador (productoPunto.asm)	5
4	Instrucciones x86 utilizadas	6
5	El rol de los registros	6
5.1	Marco de pila y parámetros	6
5.2	Registros utilizados	7
6	Ejemplo de ejecución	7
7	Repositorio GitHub	8
8	Conclusiones	8

1 Introducción

Esta práctica consiste en implementar la operación de **producto punto** (también llamado producto escalar o *dot product*) entre dos vectores de números reales, utilizando **lenguaje ensamblador x86** con la **Unidad de Punto Flotante (FPU)**. La función ensamblador es invocada desde un programa principal escrito en **C++**, que se encarga de solicitar los datos al usuario y mostrar el resultado.

El objetivo no es únicamente reproducir la operación matemática, sino entender cómo el procesador accede a arreglos en memoria mediante punteros, cómo recorre estructuras de datos con registros de índice, y cómo acumula resultados de punto flotante usando la pila interna de la FPU.

¿Qué es el producto punto?

Dados dos vectores $\vec{u} = (u_1, u_2, \dots, u_N)$ y $\vec{v} = (v_1, v_2, \dots, v_N)$ de la misma dimensión N , el producto punto se define como la suma de los productos elemento a elemento:

$$\vec{u} \cdot \vec{v} = \sum_{i=1}^N u_i \cdot v_i = u_1 v_1 + u_2 v_2 + \dots + u_N v_N \quad (1)$$

El resultado es un **escalar** (un único número), razón por la cual también se llama producto escalar. Por ejemplo, para vectores de dimensión 3:

$$(2, 3, 4) \cdot (1, 5, 2) = (2 \times 1) + (3 \times 5) + (4 \times 2) = 2 + 15 + 8 = 25 \quad (2)$$

¿Cómo se representan los vectores en memoria?

En C++, un `vector<double>` almacena sus elementos de forma **contigua** en memoria. Cada elemento de tipo `double` ocupa **8 bytes** (64 bits). Si el primer elemento está en la dirección D , los elementos están en:

Elemento	Dirección	Desplazamiento
<code>vec[0]</code>	D	+0 bytes
<code>vec[1]</code>	$D + 8$	+8 bytes
<code>vec[2]</code>	$D + 16$	+16 bytes
<code>vec[i]</code>	$D + 8i$	+8 <i>i</i> bytes

Table 1: Distribución en memoria de un arreglo de `double`

El ensamblador recorre estos elementos avanzando el puntero en 8 bytes por iteración con la instrucción `ADD ESI, 8`.

2 Organización del proyecto

El proyecto consta de dos archivos que trabajan en conjunto:

Archivo	Función
<code>productoPunto.asm</code>	Implementa en ensamblador x86 la función <code>productoPunto</code> , que recibe punteros a los dos vectores y su dimensión, y devuelve el escalar resultante.
<code>main.cpp</code>	Programa principal en C++ que solicita los datos al usuario, llena los vectores y llama a la función ensamblador.

Table 2: Archivos del proyecto

La directiva `.model flat, c` y la declaración `extern "C"` en C++ garantizan la compatibilidad de nombres entre ambos módulos. El resultado de tipo `double` se retorna en `ST(0)` de la FPU, conforme a la convención *cdecl* de x86.

El núcleo del algoritmo es un bucle de cuatro fases que se repite N veces:

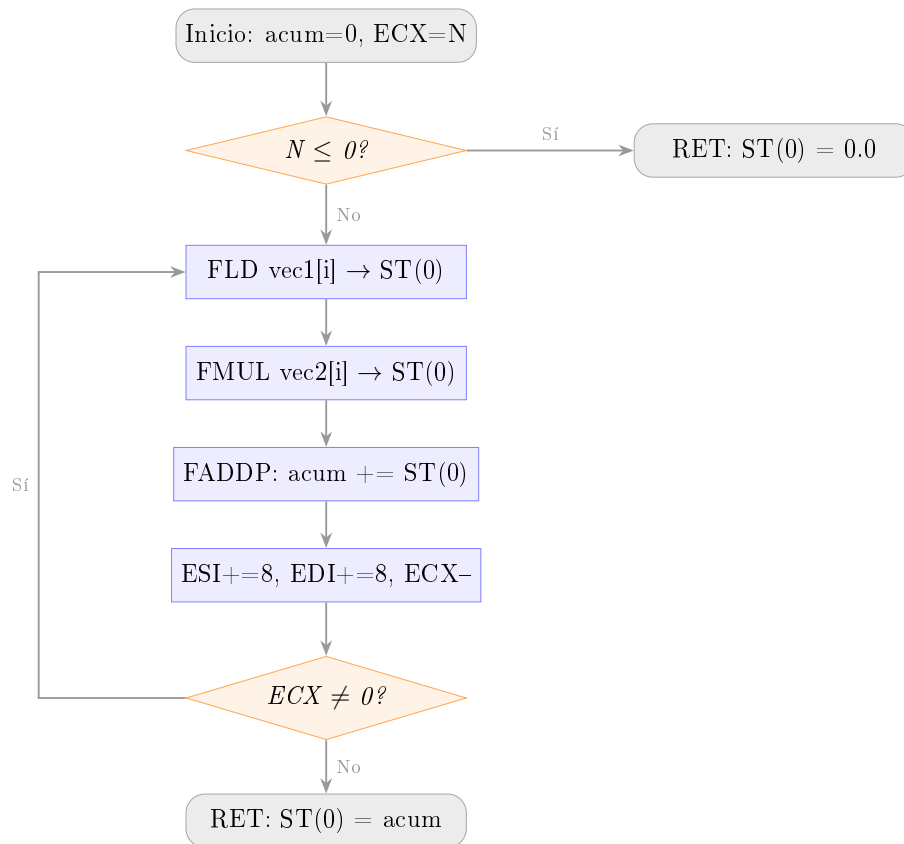


Figure 1: Diagrama de flujo del algoritmo de producto punto

Paso a paso por iteración:

1. `FLD QWORD PTR [ESI]`: carga el elemento `vec1[i]` al tope de la FPU (`ST(0)`), empujando el acumulador a `ST(1)`.
2. `FMUL QWORD PTR [EDI]`: multiplica `ST(0)` por `vec2[i]` sin hacer *pop*; el resultado (`vec1[i] × vec2[i]`) queda en `ST(0)`.
3. `FADDP ST(1), ST(0)`: suma `ST(0)` al acumulador en `ST(1)`, hace *pop* de `ST(0)`, y el acumulador actualizado queda en `ST(0)`.
4. `ADD ESI, 8 / ADD EDI, 8`: avanza ambos punteros al siguiente elemento del arreglo.
5. `DEC ECX / JNZ bucle`: decrementa el contador; si no llegó a cero, repite.

3 Código fuente

3.1 Código C++ (main.cpp)

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  extern "C" double productoPunto(double* vec1, double* vec2, int N);
6
7  int main() {
8      int n;
9      cout << "Ingresa la dimension de los vectores (N): ";
10     cin >> n;
11
12     vector<double> vec1(n);
13     vector<double> vec2(n);
14
15     cout << "\n--- DATOS DEL VECTOR 1 ---" << endl;
16     for (int i = 0; i < n; i++) {
17         cout << "Ingresa el valor para vec1[" << i << "]: ";
18         cin >> vec1[i];
19     }
20
21     cout << "\n--- DATOS DEL VECTOR 2 ---" << endl;
22     for (int i = 0; i < n; i++) {
23         cout << "Ingresa el valor para vec2[" << i << "]: ";
24         cin >> vec2[i];
25     }
26
27     double resultado = productoPunto(vec1.data(), vec2.data(), n);
28
29     cout << "\n-----\n";
30     cout << "Resultado del Producto Punto: " << resultado << endl;
31

```

```

32     return 0;
33 }

```

Listing 1: main.cpp — Programa en C++ que invoca la función ensamblador

3.2 Código ensamblador (productoPunto.asm)

```

1  ; =====
2  ; Autor: Edson Joel Carrera Avila
3  ; productoPunto.asm
4  ; =====
5
6  .586
7  .model flat, c
8
9  .code
10
11 ; Parametros: [EBP+8] = puntero a vec1 (double*)
12 ;              [EBP+12] = puntero a vec2 (double*)
13 ;              [EBP+16] = N (int)
14 productoPunto PROC
15     PUSH     EBP                ; Guardar marco de pila anterior
16     MOV      EBP, ESP          ; Establecer nuevo marco de pila
17     PUSH     ESI                ; Preservar ESI (convencion cdecl)
18     PUSH     EDI                ; Preservar EDI (convencion cdecl)
19     MOV      ESI, [EBP+8]       ; ESI = puntero al inicio de vec1
20     MOV      EDI, [EBP+12]      ; EDI = puntero al inicio de vec2
21     MOV      ECX, [EBP+16]      ; ECX = N (contador del ciclo)
22     FLDZ                     ; ST(0) = 0.0 (acumulador)
23     TEST     ECX, ECX          ; Comparar N con 0
24     JLE      fin              ; Si N <= 0, retornar 0.0
25
26 bucle:
27     FLD      QWORD PTR [ESI]    ; ST(0) = vec1[i], acum sube a ST(1)
28     FMUL     QWORD PTR [EDI]    ; ST(0) = vec1[i] * vec2[i]
29     FADDP    ST(1), ST(0)       ; ST(1) = acum + producto; pop
30     ADD      ESI, 8             ; Avanzar al siguiente double de vec1
31     ADD      EDI, 8             ; Avanzar al siguiente double de vec2
32     DEC      ECX               ; Decrementar contador
33     JNZ      bucle             ; Si ECX != 0, repetir
34
35 fin:
36     POP      EDI
37     POP      ESI
38     MOV      ESP, EBP
39     POP      EBP
40     RET
41 productoPunto ENDP
42
43 END

```

Listing 2: productoPunto.asm — Cálculo del producto punto con la FPU

4 Instrucciones x86 utilizadas

Instrucción	Qué hace
PUSH / POP	Coloca o retira un valor de 32 bits de la pila del sistema; se usan para el prólogo/epílogo y para preservar registros.
MOV	Copia un valor entre registros o entre registro y memoria.
ADD	Suma un valor inmediato a un registro; aquí avanza los punteros 8 bytes por iteración.
DEC	Decrementa el operando en 1; actualiza las banderas (<i>flags</i>) del procesador, permitiendo que JNZ las consulte.
TEST	Realiza una AND lógica entre dos operandos sin guardar el resultado; solo actualiza las banderas para verificar si ECX es cero.
JLE	Salta si el resultado de la comparación indica “menor o igual” (<i>jump if less or equal</i>); protege el bucle cuando $N \leq 0$.
JNZ	Salta si el resultado no es cero (<i>jump if not zero</i>); mantiene el bucle mientras $ECX \neq 0$.
RET	Retorna al llamador (C++); el resultado <code>double</code> queda en ST(0) de la FPU.
FLDZ	Carga la constante 0.0 al tope de la pila FPU (ST(0)); sirve para inicializar el acumulador.
FLD	Carga (<i>push</i>) un valor QWORD (8 bytes / <code>double</code>) desde memoria al tope de la pila FPU.
FMUL	Multiplica ST(0) por el operando de memoria indicado; guarda el resultado en ST(0) sin <i>pop</i> .
FADDP	Suma ST(0) a ST(i) y hace <i>pop</i> de ST(0); el resultado queda en lo que era ST(1).

Table 3: Instrucciones de propósito general utilizadas

5 El rol de los registros

5.1 Marco de pila y parámetros

Al momento en que C++ invoca `productoPunto`, coloca los tres argumentos en la pila del sistema en orden inverso (de derecha a izquierda, convención *cdecl*). Después del prólogo `PUSH EBP / MOV EBP, ESP`, el mapa de la pila queda:

Dirección	Contenido	Tipo
[EBP+16]	N	int (4 bytes)
[EBP+12]	vec2	double* (4 bytes en x86)
[EBP+8]	vec1	double* (4 bytes en x86)
[EBP+4]	dirección de retorno	—
[EBP+0]	EBP anterior	—

Table 4: Mapa del marco de pila al inicio de `productoPunto`

Los registros ESI y EDI son **preservados** (PUSH/POP) porque la convención *cdecl* exige que el llamado los restaure.

5.2 Registros utilizados

Registro	Rol	Descripción
ESI	Puntero índice fuente	Apunta al elemento actual de <code>vec1</code> ; avanza 8 bytes por iteración.
EDI	Puntero índice destino	Apunta al elemento actual de <code>vec2</code> ; avanza 8 bytes por iteración.
ECX	Contador	Almacena N y cuenta regresiva hasta 0; controla el bucle.
ST(0)	Tope de la pila FPU	Valor temporal: producto <code>vec1[i] * vec2[i]</code> en cada iteración.
ST(1)	Segundo registro FPU	Acumulador: suma parcial del producto punto.

Table 5: Registros del procesador utilizados en `productoPunto`

6 Ejemplo de ejecución

Para los vectores $\vec{u} = (2, 3, 4)$ y $\vec{v} = (1, 5, 2)$, el cálculo esperado es:

$$\vec{u} \cdot \vec{v} = (2 \times 1) + (3 \times 5) + (4 \times 2) = 2 + 15 + 8 = 25 \quad (3)$$

La tabla siguiente muestra cómo evoluciona el acumulador iteración a iteración:

Iteración	vec1[i]	vec2[i]	Producto	Acumulador
0 (inicial)	—	—	—	0
1	2.0	1.0	$2 \times 1 = 2$	2
2	3.0	5.0	$3 \times 5 = 15$	17
3	4.0	2.0	$4 \times 2 = 8$	25

Table 6: Evolución del acumulador para $\vec{u} = (2, 3, 4)$ y $\vec{v} = (1, 5, 2)$

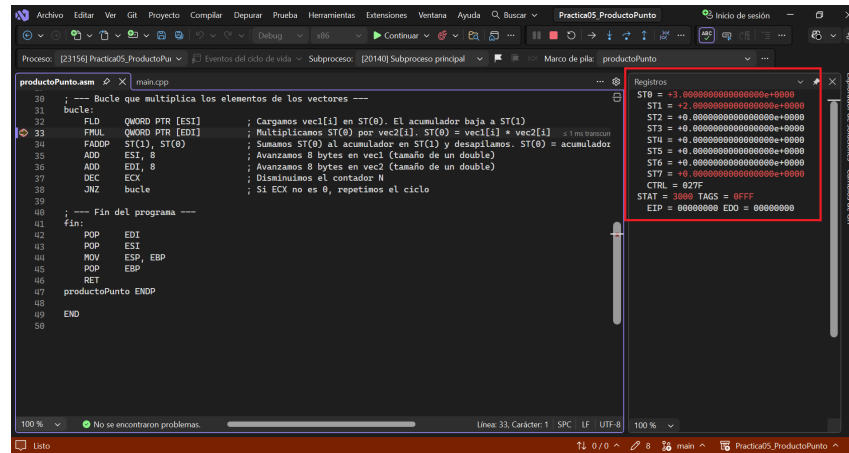
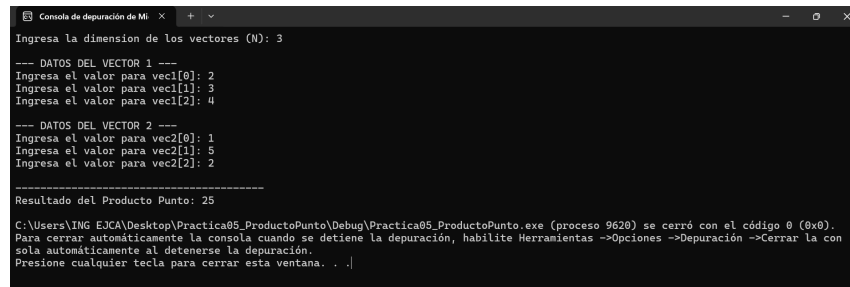


Figure 2: Registros moviéndose en tiempo real

La salida esperada en consola es:

Figure 3: Salida del programa para $\vec{u} = (2, 3, 4)$ y $\vec{v} = (1, 5, 2)$

7 Repositorio GitHub

https://github.com/7mo-ArquitecturaComputadoras/Practica05_ProductoPunto

8 Conclusiones

- El producto punto evidencia cómo el procesador **recorre arreglos en memoria mediante punteros**: avanzar ESI y EDI en 8 bytes por iteración es el equivalente ensamblador de incrementar un índice de arreglo en C++.
- El uso de **FLDZ como acumulador inicial** en la pila FPU es un patrón fundamental: cargar 0.0 antes del bucle garantiza que la primera suma sea correcta sin necesidad de un caso especial para $i = 0$.
- La instrucción **FADDP** combina suma y *pop* en un solo ciclo de reloj, manteniendo la pila FPU con profundidad constante (un elemento, el acumulador) durante todo el bucle, lo cual es una práctica eficiente de gestión de la pila de punto flotante.

- La **verificación de $N \leq 0$** con TEST y JLE antes de entrar al bucle es una práctica de programación defensiva: sin ella, un contador negativo haría que DEC / JNZ ejecutaran el bucle $2^{32} - |N|$ veces, corrompiendo la memoria.
- Preservar ESI y EDI con PUSH/POP muestra el contrato de la convención *cdecl*: el llamado es responsable de restaurar estos registros, garantizando que el código C++ no vea sus variables de iteración alteradas tras el retorno de la función ensamblador.